

Seminário FastByte Delivery: Implementação do Pilar Automação (CAMS)

1. O Conceito e o Anti-Padrão

O que é a Automação no DevOps?

No modelo CAMS (Culture, Automation, Measurement, Sharing), a **Automação** é o motor tecnológico que viabiliza a entrega rápida, frequente e segura de software. Seu principal objetivo é eliminar a intervenção manual em tarefas repetitivas e propensas a falhas humanas. Isso é realizado pela padronização dos processos de construção (build), testes e implantação (deploy) de código, formando o que chamamos de pipelines de **CI/CD (Integração e Entrega Contínuas)**. Através da automação, o software torna-se rastreável e pode ser liberado a qualquer momento, transformando processos antes estressantes em rotinas previsíveis e seguras.

O Anti-Padrão: O Caos na FastByte Delivery

O anti-padrão deste pilar é, com exatidão, o atual cenário de colapso na FastByte Delivery, caracterizado pela ausência de processos automatizados e pela dependência exclusiva do esforço braçal:

- **Deploys Manuais, Lentos e Inseguros:** A prática de atualizar o servidor arrastando arquivos via FTP ou copiando via pendrive impede qualquer tipo de auditoria, versionamento de segurança ou rollback (reversão) rápido em caso de falha.
- **A Cultura de Silos e o "Inferno do Fim de Semana":** O time de Dev cria pacotes às pressas e os "joga por cima do muro". O time de Ops recebe algo que não entende completamente, resultando em fins de semana inteiros dedicados a manter um servidor instável de pé.
- **A Síndrome do "Na Minha Máquina Funciona":** Como os ambientes (Dev, Teste, Produção) são criados e configurados manualmente, eles possuem inúmeras discrepâncias de configuração. Um código que roda perfeitamente no notebook de um desenvolvedor causa o travamento total do servidor em produção por causa de dependências ocultas não mapeadas.

2. Práticas e Ferramentas

Para solucionar a crise da FastByte, o esquadrão propõe a implementação progressiva das seguintes práticas e ferramentas de mercado, substituindo a intervenção manual pela automação estruturada:

Prática de Automação	Descrição e Como Ajuda a FastByte	Ferramentas Recomendadas
Integração Contínua (CI)	Garante que o código dos desenvolvedores seja centralizado, compilado e testado de forma automática diversas vezes ao dia. Elimina o problema de descobrir erros de compatibilidade apenas na sexta-feira.	Jenkins, GitLab CI, GitHub Actions
Entrega/Implantação Contínua (CD)	Substitui os pendrives e FTPs. O código que passou na fase de CI é automaticamente empacotado e entregue nos servidores, pronto para ser usado. Garante rastreabilidade total.	ArgoCD, AWS CodeDeploy, Spinnaker
Infraestrutura como	Transforma servidores e	Terraform, Docker,

Prática de Automação	Descrição e Como Ajuda a FastByte	Ferramentas Recomendadas
Código (IaC) e Contêinerização	configurações em arquivos de texto versionados. Com contêineres, o ambiente onde o app roda é envelopado junto com o código, extinguindo definitivamente a desculpa de que "na minha máquina funciona".	Kubernetes, Ansible
Automação de Testes	Testes de carga e unitários rodam sem intervenção humana, alertando imediatamente caso uma nova funcionalidade cause sobrecarga antes mesmo de chegar ao cliente final.	Selenium, JUnit, Cypress, Jest

3. Estudo de Caso: Amazon e a Transição para o Deploy Contínuo

O Problema no Início dos Anos 2000

A gigante Amazon já enfrentou um cenário análogo ao da FastByte Delivery. No início, sua arquitetura era um grande bloco único (monolito) e seus processos de deploy eram massivamente manuais. Atualizar o sistema exigia coordenação pesada entre Dev e Ops, processos burocráticos de aprovação e longas janelas de manutenção noturnas ou nos fins de semana, trazendo alto risco de quedas em pleno e-commerce.

A Solução Baseada na Automação

Para escalar e sobreviver, a Amazon mudou radicalmente: quebrou o sistema monolítico em pequenos serviços independentes (microsserviços) e desenvolveu uma ferramenta robusta chamada *Apollo*. O *Apollo* automatizava rigorosamente o pipeline de CI/CD para todas as equipes. O desenvolvedor passou a escrever o código e o sistema *Apollo* assumia a responsabilidade de testar e fazer a implantação na infraestrutura da Amazon sem necessidade de intervenção manual da equipe de operações.

Os Resultados e a Lição para a FastByte

O investimento em automação produziu resultados sem precedentes na indústria. Em 2011, a Amazon chocou o mercado ao revelar que, graças à automação severa, estava realizando em média **um deploy em produção a cada 11,6 segundos** (e hoje esse número é consideravelmente maior). As falhas despencaram, e reversões (rollbacks) aconteciam com um único clique, sem pânico. A lição para a FastByte Delivery é clara: para escalar o aplicativo sem destruir a saúde mental dos funcionários, os processos manuais devem ser substituídos por pipelines de automação confiáveis.

Referências Bibliográficas

- KIM, Gene; BEHR, Kevin; SPAFFORD, George. *The Phoenix Project: A Novel about IT, DevOps, and Helping Your Business Win*. IT Revolution Press, 2013.
- FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press, 2018.
- HUMBLE, Jez; FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- ALLSPAW, John; HAMMOND, Paul. *10+ Deploys Per Day: Dev and Ops Cooperation at Flickr*. O'Reilly Velocity Conference, 2009.